DAYANANDA SAGAR COLLEGE OF ENGINEERING

(An Autonomous Institute affiliated to Visvesvaraya Technological University (VTU), Belagavi,
Approved by AICTE and UGC, Accredited by NAAC with 'A' grade & ISO 9001 – 2015 Certified Institution)
Shavige Malleshwara Hills, Kumaraswamy Layout, Bengaluru-560 111, India

DEPARTMENT OF ARTIFICIAL INTELLGENCE & MACHINE LEARNING

Alternate Assessment Tool

Course Title: INTERNET OF THINGS AND 5G
Course Code: 22AI642
Semester: VI

Submitted by

P Charan – 1DS23AI403

Under the Guidance of
Dr Aruna M G
Professor
Department of Artificial Intellgence & Machine Learning
DSCE, Bengaluru

Visvesvaraya Technological University
Jnanasangama, Belagavi-590018, Karnataka, India
2025-26

Table of Contents

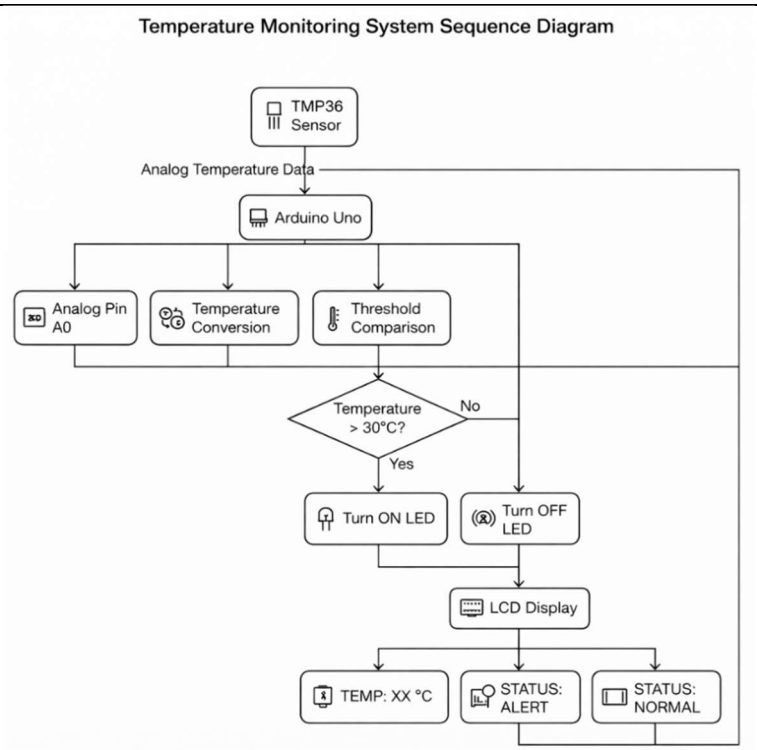
1. Tinker Andriod uno
2. Wokwi ESP
3. Wokwi STM
4.Wokwi Raspberry Pi Pico Simulator
5. TinyML

Activity 1: TinkerCAD Aurdino

<table>
<tr><td>PROGRAM STATEMENT</td><td colspan="5">Design and implement a temperature monitoring system using Arduino Uno that measures real-time temperature using a TMP36 sensor and displays the readings on an LCD. The system should continuously monitor temperature conditions and provide a visual alert using an LED when the temperature exceeds a predefined threshold, ensuring timely detection of abnormal or unsafe conditions.</td></tr>
<tr><td>AAT MARKS</td><td>Design (5)</td><td>Execution (5)</td><td>Result (5)</td><td>Innovation/ Extra Features (5)</td><td>Total</td></tr>
<tr><td></td><td></td><td></td><td></td><td></td><td></td></tr>
<tr><td>TOOLS USED</td><td colspan="5">• TinkerCAD
• Temperature Sensor
• Arduino UNO
• LED
• Resistor
• LCD Display</td></tr>
<tr><td>MAP SDG</td><td colspan="5">• SDG 3 – Good Health and Well-being
This system helps in monitoring body temperature and detecting fever conditions, supporting early diagnosis and better healthcare management.

• SDG 9 – Industry, Innovation, and Infrastructure
Demonstrates the use of embedded systems and IoT technologies to build innovative and low-cost monitoring solutions.</td></tr>
<tr><td>MAP COURSE OUTCOME of IOT</td><td colspan="5"><table>
<tr><th>CO</th><th>Mapping</th><th>Justification</th></tr>
<tr><td>CO1</td><td>Yes</td><td>Demonstrates perception layer using temperature sensor (TMP36) to monitor environmental conditions with Arduino processing.</td></tr>
<tr><td>CO2</td><td>Yes</td><td>Uses Arduino C programming and analog data acquisition to read temperature and display on LCD.</td></tr>
<tr><td>CO3</td><td>Yes</td><td>Does not include data analytics; focuses only on real-time monitoring.</td></tr>
</table></td></tr>
</table>

	CO4	Yes	Integrates temperature sensing (input) with LED alert and LCD display (output) for complete IoT-based monitoring.
MAP PROGRAM OUTCOME	**PO**	**Mapping**	**Justification**
	PO1	Yes	Applied engineering knowledge of sensors, microcontrollers, and embedded systems to build a temperature monitoring system.
	PO2	Yes	Identified the need for real-time temperature monitoring in environments and analyzed how threshold-based alerts can improve safety.
	PO3	Yes	Designed a system that displays temperature values and triggers alerts when the temperature exceeds a predefined limit.
	PO4	Yes	Investigated sensor readings and implemented logic to convert analog signals into meaningful temperature values.
	PO5	Yes	Used modern tools such as Arduino IDE and simulation platforms for implementation and testing.
	PO6	Yes	Contributes to environmental monitoring and safety by enabling early detection of abnormal temperature conditions.
MAP PROGRAM SPECIFIC OUTCOME(PSO) of DEPARTMENT	**PSO**	**Mapping**	**Justification**
	PSO 1	Yes	Demonstrates programming and problem-solving skills by implementing a temperature monitoring system using Arduino, including sensor interfacing, data processing, and threshold-based alert logic.
	PSO 2	Yes	Identifies and solves the real-world problem of environmental temperature monitoring by applying basic

			data interpretation and decision-making techniques for alert generation.
	PSO 3	Yes	Utilizes simulation tools such as Tinkercad and embedded platforms to design, test, and deploy the temperature monitoring system efficiently.
URL Link	https://www.tinkercad.com/things/0tQucg1eBRx/editel?returnTo=%2Fdashboard%2Fdesigns%2Fcircuits		
SYSTEM DIAGRAM / UML / SEQUENCE DIGRAM/ 3D Model		
COURSE COORDINATOR	Dr ARUNA M G		

Details

Abstract:

This project presents a smart temperature monitoring system using Arduino Uno, a TMP36 sensor, and an LCD display. The system continuously measures temperature and converts analog signals into real-time Celsius readings. The temperature values are displayed on a 16x2 LCD for easy monitoring. An LED indicator is used to alert when the temperature exceeds a predefined threshold, indicating abnormal or unsafe conditions. This system provides a simple, low-cost, and efficient solution for basic health and environmental monitoring applications.

Use Case:

- Patient temperature monitoring in hospitals or home care
- Fever detection and alert systems
- Smart health monitoring devices
- Environmental temperature monitoring
- IoT-based smart monitoring prototypes

Introduction:

Temperature monitoring plays a crucial role in healthcare and environmental systems. Traditional systems often lack real-time alerts and visual feedback. This project uses an Arduino Uno along with a TMP36 temperature sensor to measure temperature and display it on an LCD screen. The addition of an LED alert system enhances safety by indicating abnormal conditions. This system demonstrates how embedded systems can be used to develop simple and efficient monitoring solutions.

Objective:

- To measure temperature using TMP36 sensor
- To display real-time temperature on LCD
- To implement an alert system using LED
- To monitor temperature continuously

- To develop a low-cost health monitoring prototype

General procedure

- Connect TMP36 sensor to Arduino analog pin
- Interface 16x2 LCD with Arduino
- Read analog values from sensor
- Convert voltage into temperature
- Display temperature on LCD
- Compare with threshold value
- Trigger LED alert if temperature is high

Step to Execute:

- **Placement:** Drag an Arduino Uno, a 16x2 LCD, a TMP36 temperature sensor, and an LED with a resistor onto the workspace.
- **Powering up:** Connect the 5V pin of the Arduino to the VCC of the LCD and the left pin of the TMP36 sensor. Connect the GND pin of the Arduino to the GND, V0, and RW pins of the LCD, the short leg of the LED, and the right pin of the TMP36 sensor.
- **Data Wiring for LCD:** Connect the RS pin of the LCD to Arduino pin 12 and the E pin to pin 11. Connect the LCD data pins DB4, DB5, DB6, and DB7 to Arduino pins 5, 4, 3, and 2 respectively.
- **Sensor and Alert Wiring:** Connect the middle pin of the TMP36 sensor to analog pin A0. Connect the long leg of the LED through a resistor to digital pin 13.
- **Code and Test:** Open the code editor, switch to text mode, and paste the program. Click Start Simulation. Adjust the TMP36 sensor value and observe the LCD display changing from NORMAL to ALERT when the temperature exceeds 30 degrees Celsius.

Source Code:

```
#include <LiquidCrystal.h>
LiquidCrystal lcd(12, 11, 5, 4, 3, 2);

const int sensorPin = A0;
```

```
const int ledPin = 13;
const float threshold = 30.0;

void setup() {
  pinMode(ledPin, OUTPUT);
  lcd.begin(16, 2); // Set up the LCD's number of columns and rows

  lcd.print("BEDSIDE MONITOR");
  lcd.setCursor(0, 1);
  lcd.print("INITIALIZING...");
  delay(2000);
  lcd.clear();
}

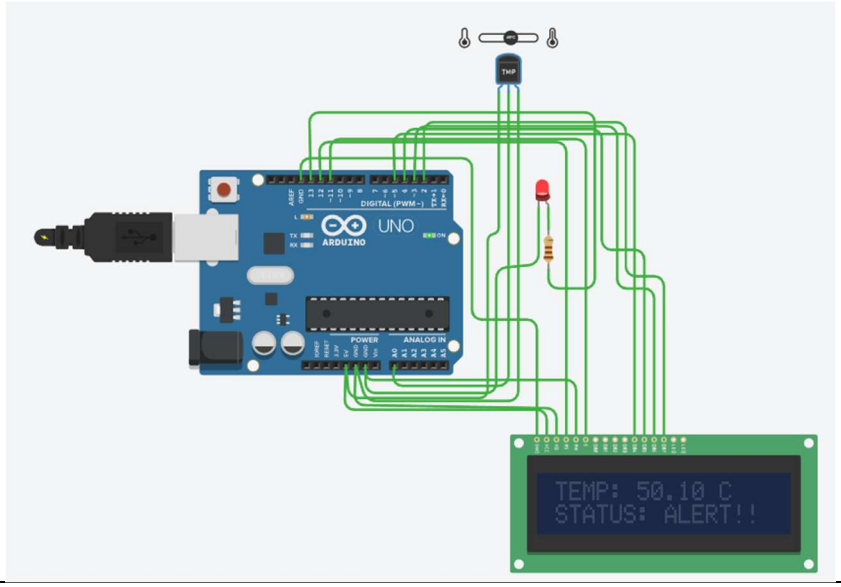
void loop() {
  int sensorVal = analogRead(sensorPin);
  float voltage = (sensorVal / 1024.0) * 5.0;
  float temperatureC = (voltage - 0.5) * 100;

  // LCD Line 1: Static Status
  lcd.setCursor(0, 0);
  lcd.print("TEMP: ");
  lcd.print(temperatureC);
  lcd.print(" C  "); // Extra spaces to clear old digits

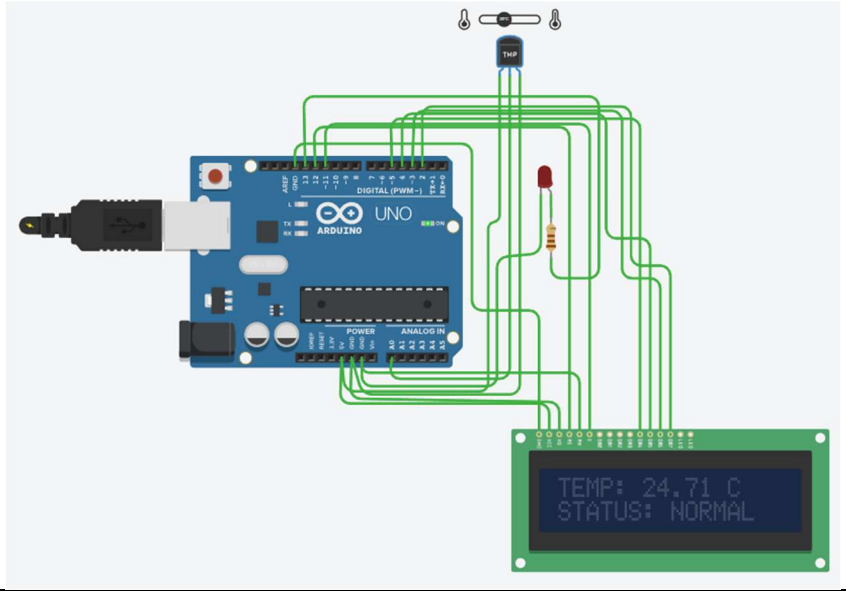
  // LCD Line 2: Dynamic Alert
  lcd.setCursor(0, 1);
  if (temperatureC > threshold) {
    digitalWrite(ledPin, HIGH);
    lcd.print("STATUS: ALERT!! ");
  } else {
    digitalWrite(ledPin, LOW);
    lcd.print("STATUS: NORMAL  ");
  }

  delay(500); // Faster refresh for smoother display
}
```

Output:



Snapshot:



Outcome:

- Successfully developed a temperature monitoring system with display
- Achieved real-time visualization using LCD
- Implemented alert mechanism using LED
- Improved system usability with clear status indication
- Demonstrated a simple and effective IoT-based monitoring solution

Activity 2: Wokwi ESP32

<table>
<tr><td>PROBLEM STATEMENT</td><td colspan="5">Design and implement a smart saline monitoring system using ESP32 that continuously measures the saline level using an ultrasonic sensor and provides real-time alerts when the level drops below a critical threshold. The system should activate visual and audio alarms and simulate an emergency response mechanism to ensure timely medical intervention and improve patient safety.</td></tr>
<tr><td>AAT MARKS</td><td>Design (5)</td><td>Execution (5)</td><td>Result (5)</td><td>Innovation/ Extra Features (5)</td><td>Total</td></tr>
<tr><td></td><td></td><td></td><td></td><td></td><td></td></tr>
<tr><td>TOOLS USED</td><td colspan="5">• Wokwi
• Ultrasonic Sensor
• LED
• Resistor
• Buzzer
• ESP 32</td></tr>
<tr><td>MAP SDG</td><td colspan="5">• SDG 3 – Good Health and Well-being
This system helps in monitoring body temperature and detecting fever conditions, supporting early diagnosis and better healthcare management.

• SDG 9 – Industry, Innovation, and Infrastructure
Demonstrates the use of embedded systems and IoT technologies to build innovative and low-cost monitoring solutions.</td></tr>
<tr><td>MAP COURSE OUTCOME of IOT</td><td colspan="5"><table>
<tr><th>CO</th><th>Mapping</th><th>Justification</th></tr>
<tr><td>CO1</td><td>Yes</td><td>Implements perception layer using ultrasonic sensor to monitor saline level.</td></tr>
<tr><td>CO2</td><td>Yes</td><td>Uses embedded programming and digital signal processing for level detection.</td></tr>
<tr><td>CO3</td><td>No</td><td>No historical analysis; only real-time alert system.</td></tr>
</table></td></tr>
</table>

	CO4	Yes	Integrates sensing, processing, and alert system (LED + buzzer + IoT).

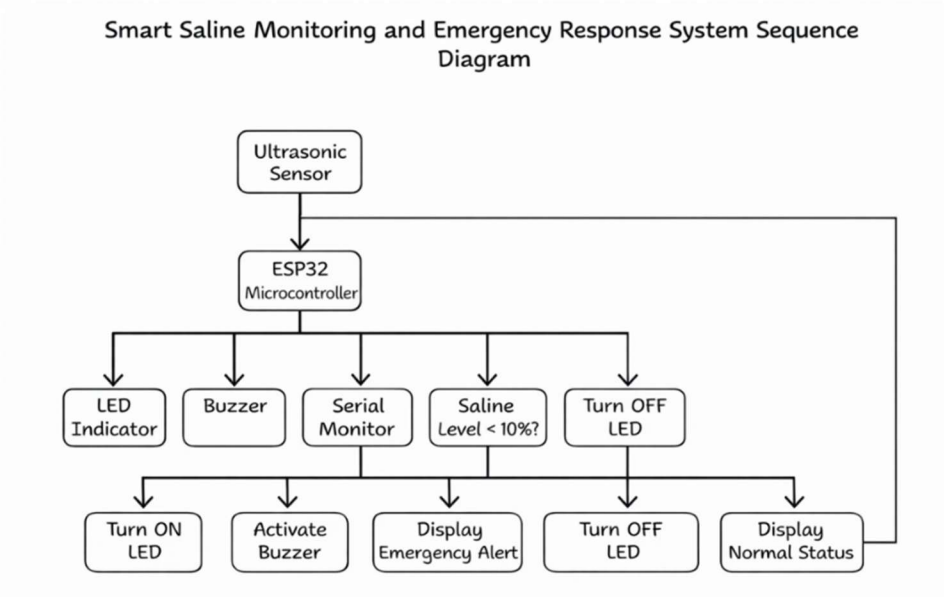
MAP PROGRAM OUTCOME

PO	Mapping	Justification
PO1	Yes	Applied knowledge of IoT, sensors, and embedded systems to develop a smart saline monitoring system.
PO2	Yes	Identified the critical healthcare issue of saline level monitoring and the risks of delayed response.
PO3	Yes	Designed an automated system that detects low saline levels and triggers alerts for immediate action.
PO4	Yes	Implemented threshold-based decision-making logic for emergency situations.
PO5	Yes	Used ESP32 platform, sensors, and simulation tools for development and testing.
PO6	Yes	Enhances healthcare systems by improving patient safety and reducing manual monitoring efforts.

MAP PROGRAM SPECIFIC OUTCOME(PSO) of DEPARTMENT

PSO	Mapping	Justification
PSO 1	Yes	Applies programming and embedded system design skills to develop an automated saline monitoring and emergency alert system using ESP32.
PSO 2	Yes	Addresses a critical healthcare problem by designing a system that detects low saline levels and triggers alerts using intelligent threshold-based logic.
PSO 3	Yes	Uses IoT platforms, simulation tools, and ESP32-based systems to implement real-time monitoring and communication.

URL Link	https://wokwi.com/projects/460845442125174785

SYSTEM DIAGRAM / UML / SEQUENCE DIGRAM/ 3D Model	
COURSE COORDINATOR	**Dr ARUNA M G**

Details

Abstract:

This project presents a Smart Saline Monitoring and Emergency Response System using ESP32 and an ultrasonic sensor (HC-SR04). The system continuously measures the saline bottle level by calculating the distance between the sensor and liquid surface. The measured value is converted into a percentage to represent the saline level. When the saline level drops below a critical threshold, the system triggers an alert using an LED and buzzer. Additionally, it simulates an emergency response by displaying a message indicating that a doctor is being notified. This system helps in preventing risks caused by unattended saline depletion in patients.

Use Case:

- Hospital patient monitoring systems
- Automatic saline level tracking
- ICU and emergency care units
- Remote healthcare monitoring
- Smart healthcare IoT solutions

Introduction:

In hospitals, monitoring saline levels manually can be inefficient and may lead to serious risks if the saline bottle becomes empty without timely intervention. This project introduces a smart solution using ESP32 and an ultrasonic sensor to automate saline level monitoring. The system continuously tracks the liquid level and alerts medical staff when the level becomes critically low. With built-in alert mechanisms and IoT capabilities, the system ensures patient safety and reduces manual effort.

Objective:

- To monitor saline level using ultrasonic sensor
- To calculate liquid level percentage in real time
- To provide alerts using LED and buzzer
- To simulate emergency response system
- To develop an IoT-based healthcare monitoring solution

General procedure

- Connect ultrasonic sensor to ESP32
- Trigger ultrasonic pulse and measure echo time

- Calculate distance and convert to saline level percentage
- Display data in Serial Monitor
- Compare level with threshold value
- Activate LED and buzzer for emergency alert

Step to Execute:

i. **Placement:** Drag an ESP32 board, an HC-SR04 ultrasonic sensor, a buzzer, and an LED with a resistor onto the workspace.

ii. **Powering up:** Connect the VCC pin of the ultrasonic sensor to the 5V pin of the ESP32 and connect the GND pin of the sensor to the GND of the ESP32. Connect the GND of the LED and buzzer to the ESP32 GND.

iii. **Data Wiring for Sensor:** Connect the TRIG pin of the ultrasonic sensor to GPIO 5 of the ESP32. Connect the ECHO pin of the ultrasonic sensor to GPIO 18 of the ESP32.

iv. **Alert Wiring:** Connect the long leg of the LED through a resistor to GPIO 2 of the ESP32. Connect the buzzer positive terminal to GPIO 4 of the ESP32.

v. **Code and Test:** Open the code editor, switch to text mode, and paste the program. Click Start Simulation. Observe the saline level percentage in the Serial Monitor. Adjust the distance in the ultrasonic sensor to simulate saline level changes and verify that the LED and buzzer activate when the saline level drops below the critical threshold.

Source Code:

```
// --- ESP32 Smart Saline & Emergency System ---
const int TRIG_PIN = 5;
const int ECHO_PIN = 18;
const int BUZZER_PIN = 4;
const int LED_PIN = 2;

void setup() {
  Serial.begin(115200);
  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
  pinMode(BUZZER_PIN, OUTPUT);
  pinMode(LED_PIN, OUTPUT);

  Serial.println("INITIALIZING SMART SALINE MONITOR...");
```

```
}
void loop() {

  // 1. Trigger the Ultrasonic pulse
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);

  // 2. Measure the response time
  long duration = pulseIn(ECHO_PIN, HIGH);

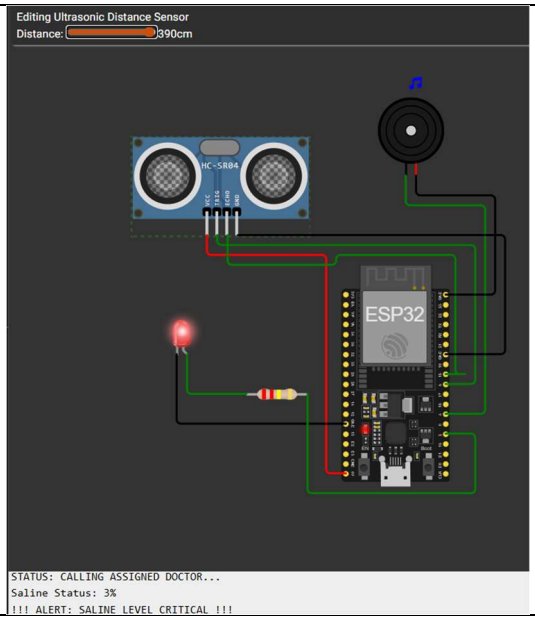
  // 3. Convert distance to Saline Level %
  // (In simulation, 400cm = Empty, 2cm = Full)
  int distance = duration * 0.034 / 2;
  int salineLevel = map(distance, 2, 400, 100, 0);
  if (salineLevel < 0) salineLevel = 0;

  // 4. Display Status
  Serial.print("Saline Status: ");
  Serial.print(salineLevel);
  Serial.println("%");

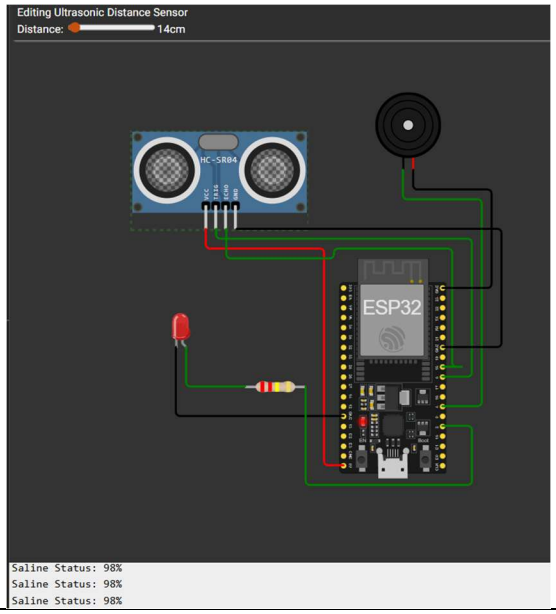
  // 5. Emergency Logic (Critical threshold < 10%)
  if (salineLevel < 10) {
    Serial.println("!!! ALERT: SALINE LEVEL CRITICAL !!!");
    Serial.println("STATUS: CALLING ASSIGNED DOCTOR...");
    digitalWrite(LED_PIN, HIGH);
    tone(BUZZER_PIN, 1000); // 1kHz alarm sound
  } else {
    digitalWrite(LED_PIN, LOW);
    noTone(BUZZER_PIN);
  }

  delay(1000);
}
```

Output:



Snapshot:



Outcome:

- Successfully developed a smart saline monitoring system
- Achieved real-time level detection using ultrasonic sensor
- Implemented emergency alert mechanism using LED and buzzer
- Reduced dependency on manual monitoring
- Demonstrated an efficient IoT-based healthcare solution

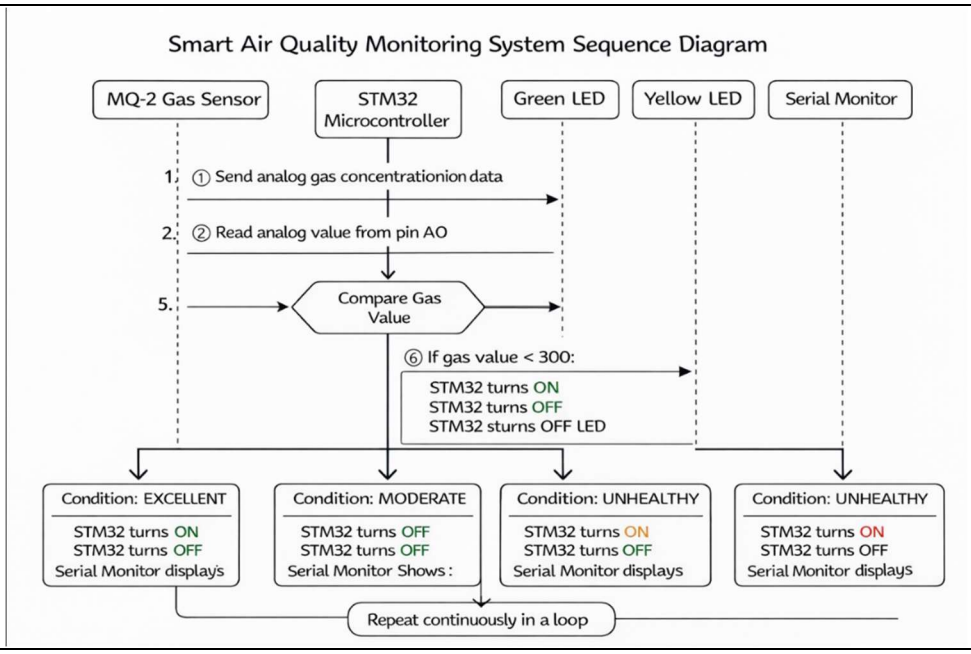
Activity 3: Wokwi STM

<table>
<tr><td>PROBLEM STATEMENT</td><td colspan="5">Design and implement an air quality monitoring system using STM32 that continuously detects gas concentration using an MQ-2 sensor and provides real-time visual indication using LEDs to classify air quality levels as safe, moderate, or dangerous.</td></tr>
<tr><td>AAT MARKS</td><td>Design (5)</td><td>Execution (5)</td><td>Result (5)</td><td>Innovation/ Extra Features (5)</td><td>Total</td></tr>
<tr><td></td><td></td><td></td><td></td><td></td><td></td></tr>
<tr><td>TOOLS USED</td><td colspan="5">• Wokwi.
• Gas Sensor
• STM Nucleo C031C6
• LED
• Resistors</td></tr>
<tr><td>MAP SDG</td><td colspan="5">• SDG 3 – Good Health and Well-being
Helps in detecting harmful gases and maintaining safe air quality, reducing health risks caused by pollution.
• SDG 11 – Sustainable Cities and Communities
Supports development of smart cities by enabling continuous monitoring of environmental conditions.
• SDG 13 – Climate Action
Contributes to awareness and monitoring of air pollution, promoting actions to improve environmental sustainability.</td></tr>
<tr><td>MAP COURSE OUTCOME of IOT</td><td colspan="5"><table><tr><th>CO</th><th>Mapping</th><th>Justification</th></tr><tr><td>CO1</td><td>Yes</td><td>DemDemonstrates the perception layer using an MQ-2 gas sensor to detect air quality and harmful gases in the environment.</td></tr></table></td></tr>
</table>

	CO2	Yes	Applies analog signal processing and embedded programming using STM32 to read sensor values and classify air quality levels.
	CO3	No	No advanced analytics or prediction is performed; the system focuses on real-time monitoring and alert indication.
	CO4	Yes	Integrates gas sensing, processing, and multi-level LED indication to implement a complete environmental monitoring system.

MAP PROGRAM OUTCOME	**PO**	**Mapping**	**Justification**
	PO1	Yes	Applied knowledge of microcontrollers and sensors for environmental monitoring
	PO2	Yes	Identified the need for air pollution monitoring to improve environmental safety.
	PO3	Yes	Designed a system that categorizes air quality into different levels using LED indicators.
	PO4	Yes	Analyzed gas sensor data and implemented threshold-based classification logic.
	PO5	Yes	Used STM32 platform and simulation tools for system implementation
	PO6	Yes	Supports environmental sustainability by monitoring air quality and raising alerts.

MAP PROGRAM SPECIFIC OUTCOME(PSO) of DEPARTMENT	**PSO**	**Mapping**	**Justification**
	PSO 1	Yes	Demonstrates programming and system design skills by developing an air quality monitoring system using STM32 and gas sensors.
	PSO 2	Yes	Solves the real-world problem of air pollution detection by analyzing sensor data and classifying air quality levels.

	PSO 3	Yes	Utilizes simulation tools and embedded platforms to design, test, and implement the air quality monitoring system.
URL Link	**https://wokwi.com/projects/460846540843282433**		
SYSTEM DIAGRAM / UML / SEQUENCE DIGRAM/ 3D Model			
COURSE COORDINATOR	**Dr ARUNA M G**		

Details

Abstract:

This project presents a Smart Air Quality Monitoring and Alert System using an STM32 microcontroller and an MQ-2 gas sensor. The system continuously reads gas concentration levels and processes the analog data to determine air quality conditions. Based on predefined thresholds, the system categorizes air quality into three levels: excellent, moderate, and unhealthy. LEDs are used to provide visual indication for each condition. The system also displays real-time values and status messages on the Serial Monitor. This solution provides a simple, low-cost, and efficient method for monitoring environmental air quality.

Use Case:

- Indoor air quality monitoring
- Industrial safety systems
- Smoke and gas leakage detection
- Smart homes and offices
- Environmental monitoring systems

Introduction:

Air pollution and gas leakage are major concerns in both industrial and residential environments. Continuous monitoring of air quality is essential to ensure safety and health. This project uses an MQ-2 gas sensor with an STM32 microcontroller to detect gas levels and provide real-time alerts. The system classifies air quality into different levels and uses LEDs for clear visual indication, making it easy to understand the environmental condition.

Objective:

- To detect gas concentration using MQ-2 sensor
- To classify air quality into multiple levels

- To provide visual alerts using LEDs
- To display real-time data in Serial Monitor
- To develop an efficient environmental monitoring system

General procedure:

- Connect MQ-2 sensor to STM32 analog pin
- Read analog gas sensor values
- Compare values with threshold ranges
- Classify air quality levels
- Control LEDs based on condition
- Display results in Serial Monitor

Step to Execute:

- **Placement:** Drag an STM32 Nucleo board, an MQ-2 gas sensor, three LEDs, and resistors onto the workspace.
- **Powering up:** Connect the VCC pin of the MQ-2 sensor to the 5V pin of the STM32 and connect the GND pin of the sensor to the GND of the STM32. Connect the GND of all LEDs to the STM32 GND.
- **Data Wiring for Sensor:** Connect the analog output pin of the MQ-2 sensor to analog pin A0 of the STM32.
- **Indicator Wiring:** Connect the green LED through a resistor to pin D2 of the STM32. Connect the yellow LED through a resistor to pin D3. Connect the red LED through a resistor to pin D4.
- **Code and Test:** Open the code editor, switch to text mode, and paste the program. Start the simulation. Observe gas sensor values in the Serial Monitor. Adjust sensor input to simulate gas levels and verify that LEDs change according to air quality conditions.

Source Code:

```
// STM32 + MQ-2 Gas Sensor Air Quality Monitor
const int mq2Pin = A0;
const int greenLed = D2;
const int yellowLed = D3;
const int redLed = D4;

void setup() {
  Serial.begin(115200);
  pinMode(greenLed, OUTPUT);
  pinMode(yellowLed, OUTPUT);
  pinMode(redLed, OUTPUT);

  Serial.println("STM32 MQ-2 Air Quality System Initializing...");
  Serial.println("Status: Sensor Heating Up...");
  delay(2000); // Simulated preheat time
}

void loop() {
  int airQualityValue = analogRead(mq2Pin);

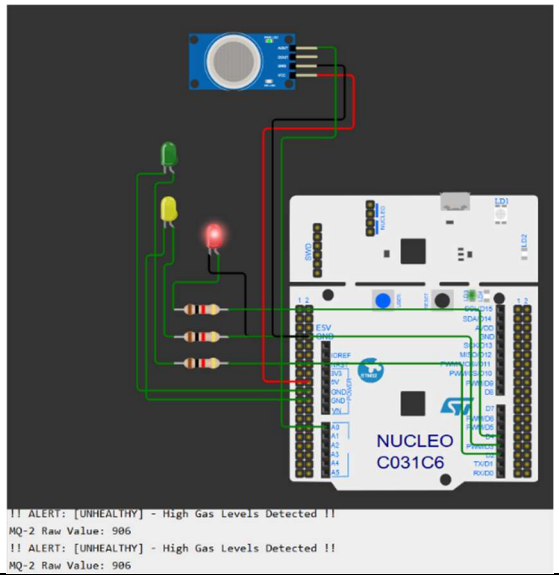
  Serial.print("MQ-2 Raw Value: ");
  Serial.println(airQualityValue);

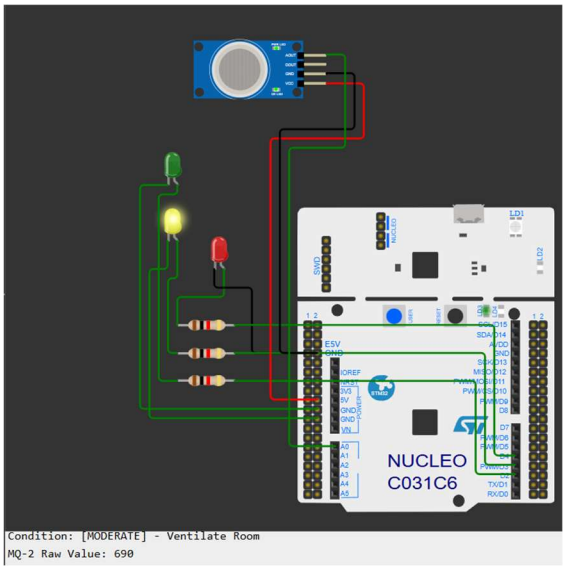
  // Thresholds for Gas/Smoke Detection
  if (airQualityValue < 300) {
    // Stage 1: Clean Air
    digitalWrite(greenLed, HIGH);
    digitalWrite(yellowLed, LOW);
    digitalWrite(redLed, LOW);
    Serial.println("Condition: [EXCELLENT]");
  }
  else if (airQualityValue >= 300 && airQualityValue < 700) {
    // Stage 2: Moderate Pollution/Comfort decrease
    digitalWrite(greenLed, LOW);
    digitalWrite(yellowLed, HIGH);
    digitalWrite(redLed, LOW);
    Serial.println("Condition: [MODERATE] - Ventilate Room");
  }
```

```
  else {
    // Stage 3: High Gas/Smoke Concentration
    digitalWrite(greenLed, LOW);
    digitalWrite(yellowLed, LOW);
    digitalWrite(redLed, HIGH);
    Serial.println("!! ALERT: [UNHEALTHY] - High Gas Levels Detected !!");
  }

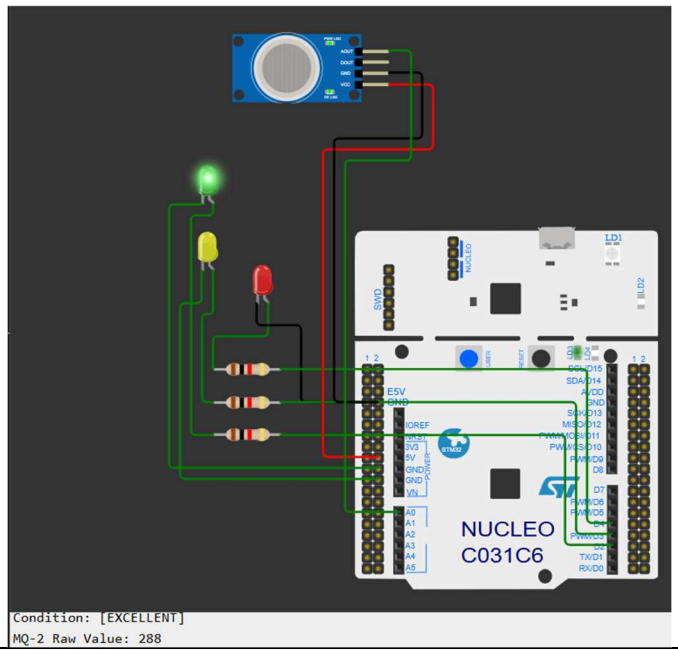
  delay(1000);
}
```

Output:





Snapshot:



Outcome:

- Successfully developed an air quality monitoring system
- Achieved real-time detection of gas concentration
- Implemented multi-level alert system using LEDs
- Demonstrated an effective environmental monitoring solution
- Improved safety awareness with clear visual indicators

Activity 4: Wokwi RaspberryPi Pico simulator

<table>
<tr><td>PROBLEM STATEMENT</td><td colspan="5">Design and implement a smart heart rate monitoring system using Raspberry Pi Pico that reads analog pulse data, converts it into BPM (beats per minute), displays the value on an LCD, and triggers an alert using a buzzer when the heart rate exceeds a safe threshold.</td></tr>
<tr><td>AAT MARKS</td><td>Design (5)</td><td>Execution (5)</td><td>Result (5)</td><td>Innovation/ Extra Features (5)</td><td>Total</td></tr>
<tr><td></td><td></td><td></td><td></td><td></td><td></td></tr>
<tr><td>TOOLS USED</td><td colspan="5">• Wokwi
• LCD display
• Buzzer
• Raspberry Pi Pico
• Potentiometer</td></tr>
<tr><td>MAP SDG</td><td colspan="5">• SDG 3 – Good Health and Well-being
This system helps in monitoring body Heartrate and detecting conditions, supporting early diagnosis and better healthcare management.

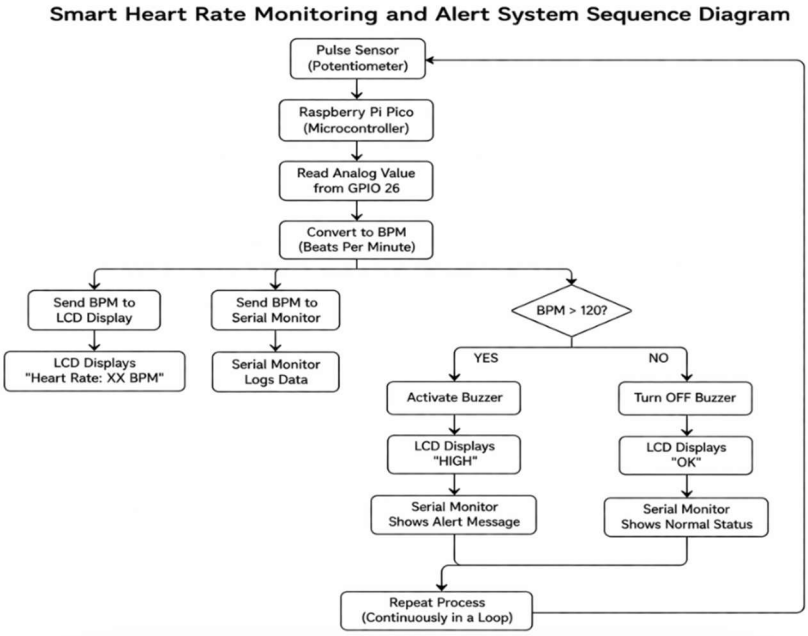
• SDG 9 – Industry, Innovation, and Infrastructure
Demonstrates the use of embedded systems and IoT technologies to build innovative and low-cost monitoring solutions.</td></tr>
<tr><td>MAP COURSE OUTCOME of IOT</td><td colspan="5"><table>
<tr><th>CO</th><th>Mapping</th><th>Justification</th></tr>
<tr><td>CO1</td><td>Yes</td><td>Implements perception layer using a pulse sensor to measure heart rate and monitor patient health conditions in real time.</td></tr>
<tr><td>CO2</td><td>Yes</td><td>Uses MicroPython programming and analog signal processing to convert sensor readings into BPM values.</td></tr>
<tr><td>CO3</td><td>No</td><td>Does not include predictive analytics; focuses only on real-time monitoring and alert generation.</td></tr>
</table></td></tr>
</table>

	CO4	Yes	Integrates sensing (pulse sensor), processing (Pico), and output (LCD and buzzer) for a complete health monitoring system.

MAP PROGRAM OUTCOME	PO	Mapping	Justification
	PO1	Yes	Applied knowledge of embedded systems and biomedical sensors.
	PO2	Yes	Identified the need for continuous heart rate monitoring in healthcare.
	PO3	Yes	Designed a system that displays BPM and generates alerts when abnormal conditions are detected.
	PO4	Yes	Processed analog signals and implemented logic for threshold-based alerting.
	PO5	Yes	Used Raspberry Pi Pico and simulation tools for implementation.
	PO6	Yes	Improves healthcare monitoring and patient safety.

MAP PROGRAM SPECIFIC OUTCOME(PSO) of DEPARTMENT	PSO	Mapping	Justification
	PSO 1	Yes	Applies programming and problem-solving skills to develop a heart rate monitoring system using Raspberry Pi Pico and sensor data processing.
	PSO 2	Yes	Addresses healthcare monitoring challenges by detecting abnormal heart rate conditions and generating alerts.
	PSO 3	Yes	Uses embedded platforms and simulation tools to design and implement a real-time health monitoring system.

URL Link	https://wokwi.com/projects/460850902742046721

SYSTEM DIAGRAM / UML / SEQUENCE DIGRAM/ 3D Model	
COURSE COORDINATOR	**Dr ARUNA M G**

Details

Abstract:

This project presents a Smart Heart Rate Monitoring and Alert System using Raspberry Pi Pico, an analog pulse sensor (simulated using a potentiometer), an I2C LCD display, and a buzzer. The system continuously reads analog signals and converts them into BPM values to represent heart rate. The measured values are displayed in real time on the LCD for easy monitoring. When the heart rate exceeds a predefined threshold, the system activates a buzzer alert to indicate abnormal conditions. This system provides a simple, low-cost, and efficient solution for basic health monitoring applications.

Use Case:

- Patient heart rate monitoring
- Fitness and health tracking devices
- Remote health monitoring systems
- Wearable healthcare solutions
- Early detection of abnormal heart conditions

Introduction:

Monitoring heart rate is essential for assessing a person's health condition. Traditional systems may not provide real-time alerts or continuous monitoring. This project uses Raspberry Pi Pico along with a pulse sensor to measure heart rate and display it on an LCD. The system also includes a buzzer alert mechanism to notify when abnormal heart rates are detected. This makes it suitable for basic healthcare and IoT-based monitoring systems.

Objective:

- To measure heart rate using analog input
- To convert sensor data into BPM values
- To display real-time heart rate on LCD
- To trigger alert using buzzer
- To develop a low-cost health monitoring system

General procedure

- Connect pulse sensor (potentiometer) to analog pin
- Interface I2C LCD with Raspberry Pi Pico
- Read analog values continuously
- Convert values into BPM using mapping
- Display BPM on LCD
- Compare BPM with threshold value
- Activate buzzer if BPM exceeds limit

Step to Execute:

- **Placement:** Drag a Raspberry Pi Pico, an I2C LCD display, a potentiometer, and a buzzer onto the workspace.
- **Powering up:** Connect the VCC pin of the LCD and potentiometer to the 3.3V pin of the Pico. Connect all GND pins of the LCD, potentiometer, and buzzer to the GND of the Pico.
- **Data Wiring for LCD:** Connect the SDA pin of the LCD to GPIO 4 and the SCL pin to GPIO 5 of the Pico.
- **Sensor Wiring:** Connect the output pin of the potentiometer to analog pin GPIO 26 of the Pico.
- **Alert Wiring:** Connect the positive terminal of the buzzer to GPIO 15 of the Pico.
- **Code and Test:** Open the code editor, switch to text mode, and paste the program. Start the simulation. Adjust the potentiometer to simulate heart rate changes and observe BPM values on the LCD. Verify that the buzzer activates when BPM exceeds the threshold.

Source Code:

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

// Try 0x27 first, if not working change to 0x3F
LiquidCrystal_I2C lcd(0x27, 16, 2);

const int pulsePin = 26;   // Potentiometer (ADC)
const int buzzerPin = 15;
```

```
void setup() {
  Serial.begin(115200);
  pinMode(buzzerPin, OUTPUT);

  // IMPORTANT for Raspberry Pi Pico
  Wire.setSDA(4);   // GP4
  Wire.setSCL(5);   // GP5
  Wire.begin();

  // Initialize LCD
  lcd.init();
  lcd.backlight();

  lcd.setCursor(0, 0);
  lcd.print("Health Monitor");
  lcd.setCursor(0, 1);
  lcd.print("Initializing...");
  delay(2000);
}

void loop() {
  int rawVal = analogRead(pulsePin);

  // Map ADC value to BPM
  int bpm = map(rawVal, 0, 1023, 60, 160);

  // Display on LCD
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("Heart Rate:");

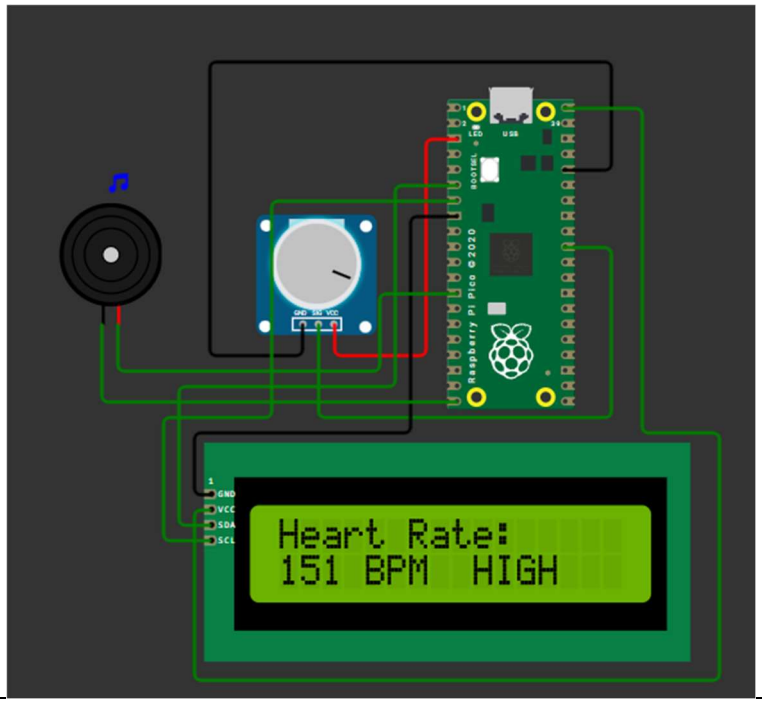
  lcd.setCursor(0, 1);
  lcd.print(bpm);
  lcd.print(" BPM");

  // Serial log
  Serial.print("BPM: ");
  Serial.println(bpm);
```

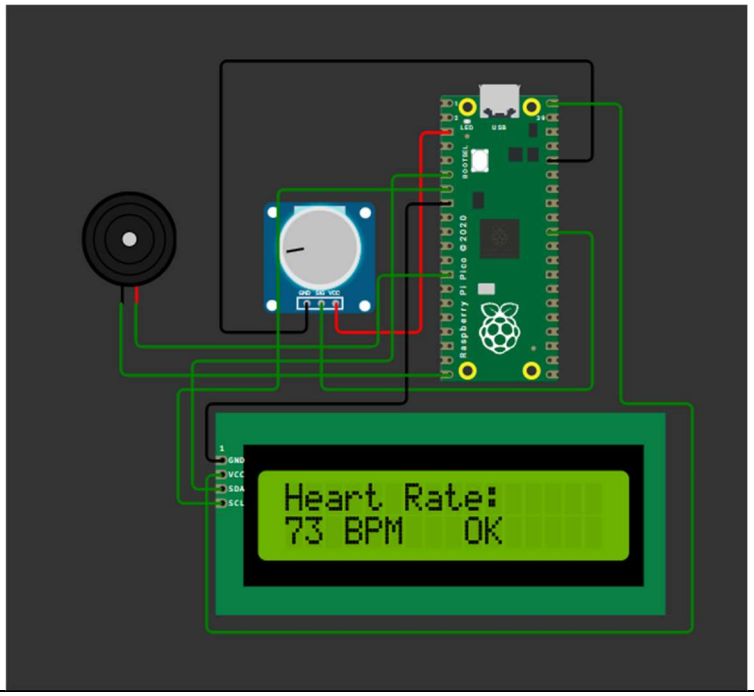
```
  // Alert system
  if (bpm > 120) {
    lcd.setCursor(9, 1);
    lcd.print("HIGH");
    tone(buzzerPin, 1000);
  } else {
    lcd.setCursor(9, 1);
    lcd.print("OK  ");
    noTone(buzzerPin);
  }

  delay(1000);
}
```

Output:



Snapshot:



Outcome:

- Successfully developed a heart rate monitoring system
- Achieved real-time BPM calculation and display
- Implemented alert system using buzzer
- Demonstrated a basic IoT-based healthcare solution
- Improved system usability with real-time feedback

Activity 5: TinyML simulation in Wokwi

<table>
<tr><td>PROBLEM STATEMENT</td><td colspan="5">In smart environments, there is a need for simple and efficient human-machine interaction methods. Traditional switches require physical contact, which may not always be convenient.</td></tr>
<tr><td>AAT MARKS</td><td>Design (5)</td><td>Execution (5)</td><td>Result (5)</td><td>Innovation/ Extra Features (5)</td><td>Total</td></tr>
<tr><td></td><td></td><td></td><td></td><td></td><td></td></tr>
<tr><td>TOOLS USED</td><td colspan="5">• Wokwi
• ESP32 DevKit V1
• Microphone Sound Sensor Module
• LED</td></tr>
<tr><td>MAP SDG</td><td colspan="5">• SDG 9: Industry, Innovation and Infrastructure
Promotes innovation through smart and interactive systems using IoT and AI.

• SDG 11: Sustainable Cities and Communities
Supports development of smart homes and automated environments.</td></tr>
<tr><td>MAP COURSE OUTCOME of IOT</td><td colspan="5"><table>
<tr><th>CO</th><th>Mapping</th><th>Justification</th></tr>
<tr><td>CO1</td><td>Yes</td><td>Demonstrates perception layer of IoT architecture using input sensors (such as IR sensors, touch sensors, or gesture modules) to detect user actions, which are processed by a microcontroller</td></tr>
<tr><td>CO2</td><td>Yes</td><td>Uses Arduino C programming, sensor interfacing, and digital input processing to enable contactless control of devices through programmed logic</td></tr>
<tr><td>CO3</td><td>No</td><td>Data analytics is not involved as the system focuses on real-time interaction and control rather than data storage or analysis</td></tr>
<tr><td>CO4</td><td>Yes</td><td>Integrates input sensing (gesture/touch/IR detection) and output actuation (switching relays, lights, or appliances) to implement a complete sense-process-act IoT system</td></tr>
</table></td></tr>
</table>

MAP PROGRAM OUTCOME

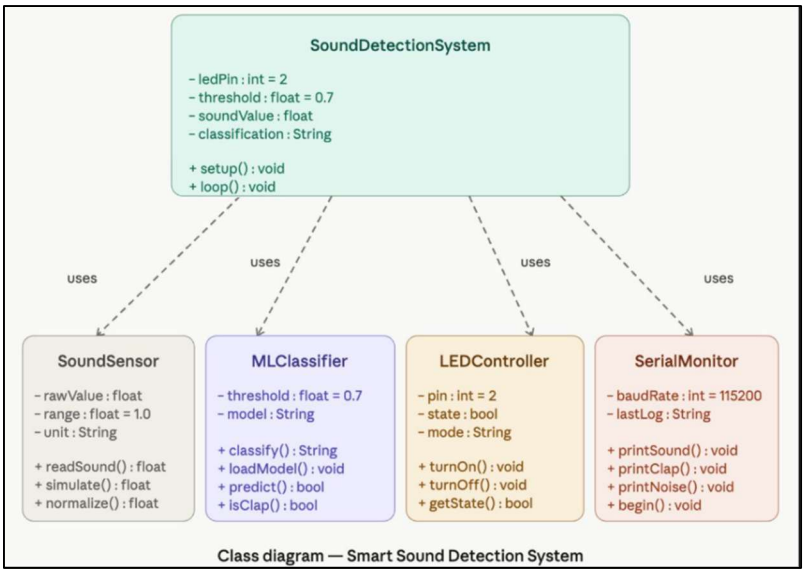
PO	Mapping	Justification
PO1	Yes	Applied knowledge of sensors and embedded systems for developing touchless or automated control mechanisms
PO2	Yes	Identified and analyzed the limitation of traditional physical switches in modern smart environments
PO3	Yes	Designed a convenient human-machine interaction system to enable contactless or automated control
PO5	Yes	Used modern engineering tools such as microcontrollers and simulation/software platforms for system development
PO6	Yes	Improves user convenience and accessibility, supporting safer and more hygienic interaction in smart environments

MAP PROGRAM SPECIFIC OUTCOME (PSO) of DEPARTMENT

PSO	Mapping	Justification
PSO 1	Yes	Demonstrates programming and problem-solving by implementing sensor-based or input-driven control logic for touchless human–machine interaction
PSO 2	Yes	Addresses the real-world need for convenient and contactless control in smart environments by reducing dependence on physical switches
PSO 3	Yes	Utilizes embedded systems or IoT simulation tools to design, test, and validate a smart interaction-based control system

URL Link

https://wokwi.com/projects/460938409570791425

SYSTEM DIAGRAM / UML / SEQUENCE DIGRAM/ 3D Model	
COURSE COORDINATOR	Dr ARUNA M G

Details

Abstract:

This project presents a Smart Sound Detection System that distinguishes between clap sounds and background noise. It uses simulated or real audio data to classify sound signals. Based on the detection, the system can trigger actions such as turning devices ON or OFF.

Use Case:

Smart home automation with clap-based control and touchless switches enables intuitive, hands-free operation of devices for greater convenience and hygiene. Assistive technology systems enhance accessibility for users with disabilities.

Introduction:

Human interaction with devices is evolving towards more natural and touchless methods. Sound-based control is one such approach. This project uses sound intensity or classification techniques to distinguish between clap and noise. By identifying specific sound patterns, the system can trigger actions, making it useful for smart environments.

Objective:

- To detect sound signals
- To classify clap and noise
- To implement simple AI or threshold-based logic
- To enable touchless control systems

General procedure

- Collect sound data (clap and noise)
- Train classification model (optional using Edge Impulse)
- Deploy model to microcontroller
- Read sound input (sensor/simulated)
- Analyze sound intensity or pattern

- Display classification result

Step to Execute:

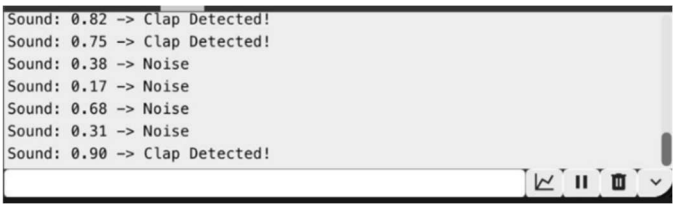
- Create project in Edge Impulse (optional)
- Upload audio samples (clap and noise)
- Train audio classification model
- Deploy model as Arduino library
- Open Wokwi and select ESP32
- Paste code and run simulation
- Observe output in Serial Monitor

Source Code:

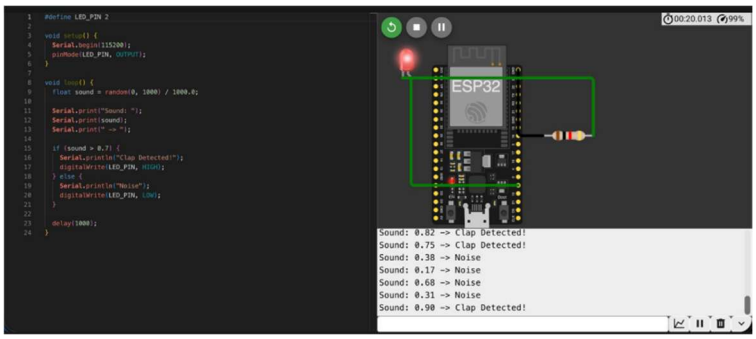
```
#include <TensorFlowLite.h>
#include "model.h"
#include "tensorflow/lite/micro/all_ops_resolver.h"
#include "tensorflow/lite/micro/micro_interpreter.h"
#define LED_PIN 2
#define SOUND_PIN 34
tflite::MicroInterpreter* interpreter;
TfLiteTensor* input;
TfLiteTensor* output;
uint8_t tensor_arena[4096];
void setup() {
  Serial.begin(115200);
  pinMode(LED_PIN, OUTPUT);
  const tflite::Model* model = tflite::GetModel(model_tflite);
  static tflite::AllOpsResolver resolver;
  static tflite::MicroInterpreter static_interpreter(
      model, resolver, tensor_arena, 4096);
  interpreter = &static_interpreter;
  interpreter->AllocateTensors();
  input = interpreter->input(0);
  output = interpreter->output(0);
}
void loop() {
  float sound = analogRead(SOUND_PIN) / 4095.0;
  input->data.f[0] = sound;
  interpreter->Invoke();
  if (output->data.f[0] > 0.8) {
    Serial.println("Clap Detected!");
```

```
    digitalWrite(LED_PIN, HIGH);
  } else {
    Serial.println("Noise");
    digitalWrite(LED_PIN, LOW);
  }
  delay(1000);
}
```

Output:



Snapshot:



Outcome:

The Smart Sound Detection System successfully distinguishes between clap and background noise.

It demonstrates the use of IoT and basic AI for touchless control and smart automation.